

República Bolivariana de Venezuela
Ministerio del poder popular para la Defensa
Universidad nacional experimental politécnica de la fuerza armada
UNEFA – Sede “San Felipe”

PRÁCTICA DE LABORATORIO — SISTEMAS OPERATIVOS
Unidades V y VI: Gestión de E/S y Entornos de Virtualización

Ing. Maille Altuve

ING. De sistemas

Sistemas operativos

VI Semestre

Gabriel Alejos C.I 31.325.236

Josué Vílchez C.I 31.342.077

10-06-26

1. Descripción y Temas Cubiertos

Profundizarán en la abstracción de dispositivos de caracteres del kernel, trabajando directamente con `/dev/null`, `/dev/zero`, `/dev/random` y `/dev/urandom`. El diferenciador de virtualización es la comparación entre Full Clone y Linked Clone, analizando cómo el hypervisor gestiona el almacenamiento compartido mediante Copy-on-Write.

Temas de las Unidades V y VI que cubre esta práctica:

- 5.1 Diferencia fundamental entre dispositivos de caracteres y de bloque
- 5.2 Buffering en dispositivos de caracteres: sin buffer interno
- 5.3 Estructura de un driver de caracteres: major/minor numbers, struct file_operations
- 5.4 El sistema de archivos `/dev` como abstracción: udev y devtmpfs
- 5.5 Manejo de errores: SIGPIPE, bloqueo en `/dev/random`, EOF en `/dev/null`
- 6.1 Cómo VirtualBox abstrae dispositivos del anfitrión al huésped
- 6.2 Copy-on-Write en Linked Clone: compartición de bloques de disco

Enfoque exclusivo de esta pareja:

Subsistema de E/S principal: E/S de Dispositivos de Caracteres (`/dev`)

Experimento de virtualización: Full Clone vs Linked Clone en VirtualBox

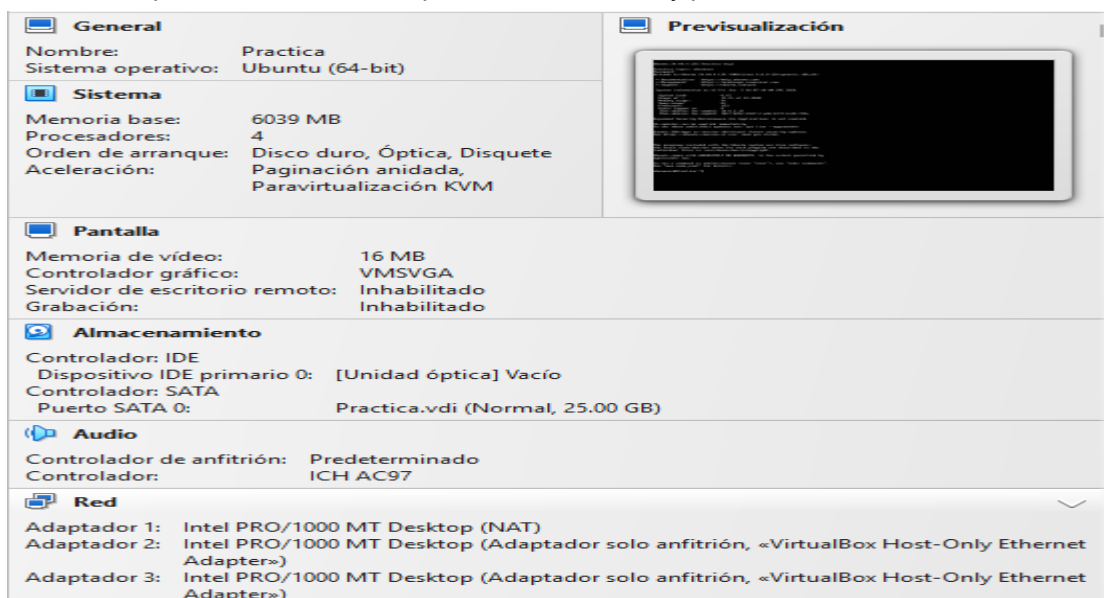
2. Actividades Base — Obligatorias para Todas las Parejas

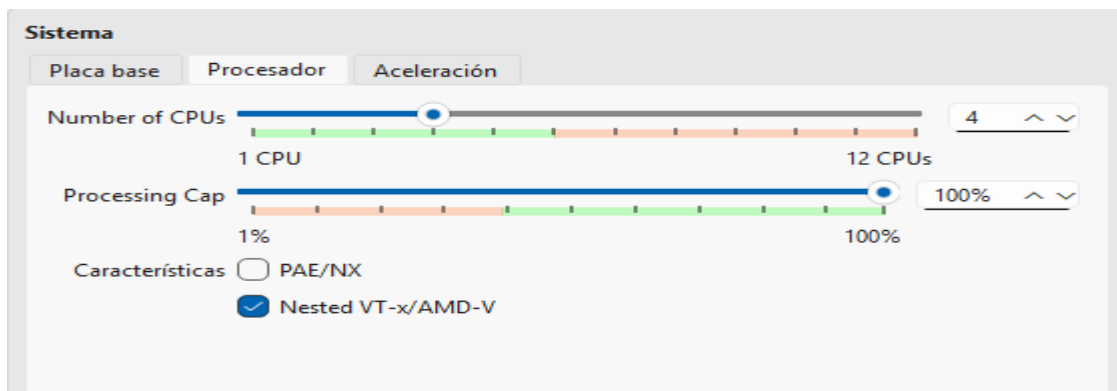
Completar estas secciones *ANTES* del proyecto integrador. Incluir capturas de *AMBOS* equipos.

A. Preparación de la VM (Temas 5.1 y 6.1)

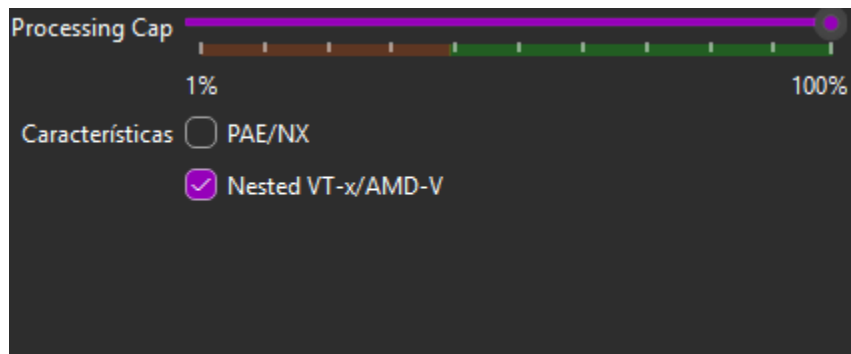
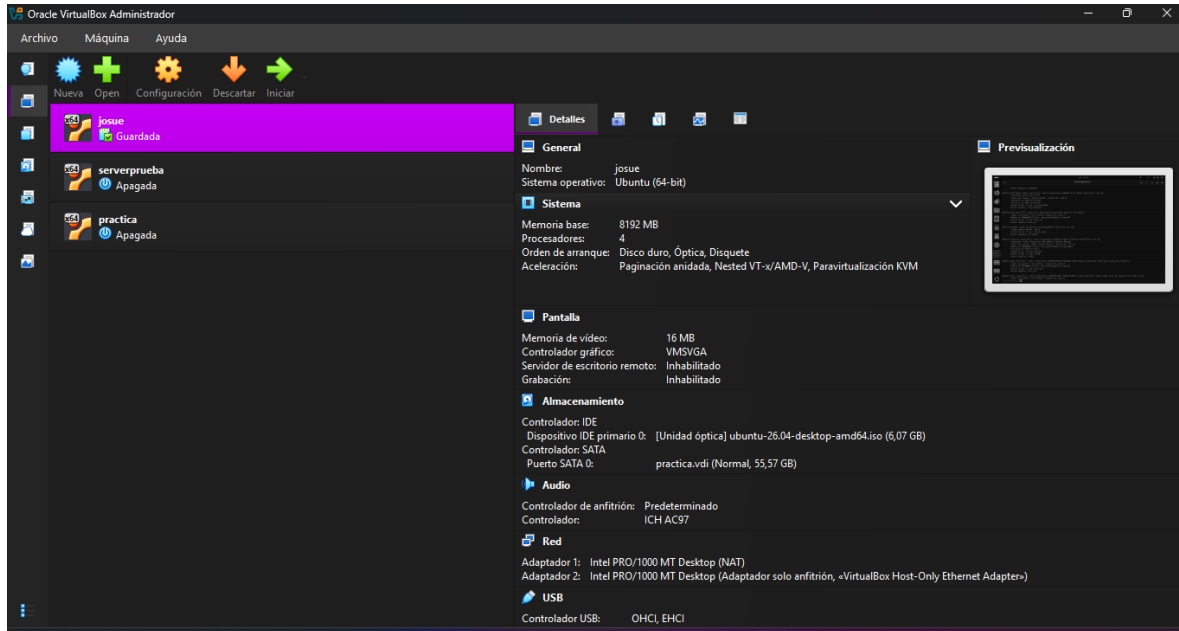
Configurar la VM con los parámetros indicados. Documentar con captura de la pantalla de configuración de VirtualBox:

- SO: Ubuntu Server 24.04 LTS — RAM: 2 GB mínimo — Disco: 20 GB dinámico
- CPU: 2 núcleos con VT-x / AMD-V habilitado (pestaña System > Processor)
- Red: Adaptador 1 en NAT + Adaptador 2 en Host-Only para acceso SSH desde el anfitrión





MAQUINA VIRTUAL GABRIEL ALEJOS



MAQUINA VIRTUAL JOSUE VILCHEZ

Clasificación del hipervisor (Tema 6.1) — responder con evidencia de comandos:

1. Dentro de la VM: `systemd-detect-virt` y `lscpu | grep -i hypervisor`
2. En el ANFITRIÓN: `lsmod | grep vbox` y `modinfo vboxdrv` (documentar qué hace este módulo)



```

vboxuser@Practica:~$ modinfo vboxdrv
filename:       /lib/modules/6.8.0-124-generic/updates/dkms/vboxdrv.ko.zst
version:        7.0.16_Ubuntu r162802 (0x00330004)
license:        GPL
description:     Oracle VM VirtualBox Support Driver
author:         Oracle and/or its affiliates
srcversion:     D5D0C77EB03C410BBA1077C
depends:
retpoline:      Y
name:           vboxdrv
vermagic:       6.8.0-124-generic SMP preempt mod_unload modversions
parm:           disabled:Disable automatic module loading (int)
parm:           force_async_tsc:force the asynchronous TSC mode (int)
vboxuser@Practica:~$

```

El módulo vboxdrv es el encargado de hacer en enlace entre nuestra maquina principal y nuestra maquina virtual, se encarga de gestionar recursos como CPU y memoria para garantizar que el sistema virtualizado funcione de manera fluida y estable.

3. Conclusión justificada: ¿VirtualBox es Tipo 1 o Tipo 2? ¿Qué evidencia lo demuestra?

VirtualBox es un hipervisor de tipo 2, esto se debe a que no se instala directamente dentro del hardware de la computadora, al contrario, esta trabaja como una aplicación dentro de un sistema operativo ya creado, la mayor ventaja de este tipo de hipervisores es su estabilidad para hacer pruebas de un sistema operativo y el desarrollo de software, sin embargo, su rendimiento se ve reducido debido a la capa entre el hardware y el sistema operativo que esta virtualizado.

B. Exploración del Subsistema de E/S desde /proc y /sys (Tema 5.1)

Para cada comando: tomar captura y escribir 2 oraciones explicando qué muestra en términos de E/S:

1. cat /proc/interrupts — dispositivos que generan interrupciones y cuántas por CPU

```

vboxuser@Practica:~$ cat /proc/interrupts
0:          CPU0          CPU1          CPU2          CPU3          IO-APIC  2-edge    timer
1:          0            0            0            0            88      IO-APIC  1-edge    i8042
8:          0            0            0            0            0       IO-APIC  8-edge    rtc0
9:          0            0            0            0            0       IO-APIC  9-fasteoi acpi
12:         0           158            0            0            0       IO-APIC 12-edge    i8042
14:         0            0           770            0            0       IO-APIC 14-edge    ata_piix
15:         0            0            0            0            0       IO-APIC 15-edge    ata_piix
18:         0            0            0            0            0       IO-APIC 18-fasteoi vmwgfx
19:         0            0           265            0            0       IO-APIC 19-fasteoi ehci_hcd:usb2, enp0s3
20:         0            0            0            0            0       IO-APIC 20-fasteoi vboxguest
21:         0           4911            0            0            0       IO-APIC 21-fasteoi ahci[0000:00:0d.0], snd_intel8x0
22:         24            0            0            0            0       IO-APIC 22-fasteoi ohci_hcd:usb1
NMI:         0            0            0            0            0   Non-maskable interrupts
LOC:    90061           7951          10609           9640   Local timer interrupts
SPU:         0            0            0            0            0   Spurious interrupts
PMI:         0            0            0            0            0   Performance monitoring interrupts
IWI:         0            0           17            1           12   IRQ work interrupts
RTR:         0            0            0            0            0   APIC ICR read retries
RES:        208           253           282           238   Rescheduling interrupts
CAL:        9049          8402          7258          6661   Function call interrupts
TLB:         24            27            38            28   TLB shutdowns
TRM:         0            0            0            0            0   Thermal event interrupts
THR:         0            0            0            0            0   Threshold APIC interrupts
DFR:         0            0            0            0            0   Deferred Error APIC interrupts
MCE:         0            0            0            0            0   Machine check exceptions
MCP:         3            3            3            3            3   Machine check polls
ERR:         0
MIS:         0
PIN:         0            0            0            0            0   Posted-interrupt notification event
NPI:         0            0            0            0            0   Nested posted-interrupt event
PIW:         0            0            0            0            0   Posted-interrupt wakeup event
vboxuser@Practica:~$

```

```

ubuntu@ubuntu:~$ cat /proc/interrupts
bash: cat/proc/interrupts: No such file or directory
ubuntu@ubuntu:~$ cat /proc/interrupts
0:          CPU0          CPU1          CPU2          CPU3          IO-APIC  2-edge    timer
1:          0            0            0          547      IO-APIC  1-edge    i8042
8:          0            0            0            0            0       IO-APIC  8-edge    rtc0
9:          0            0            0            0            0       IO-APIC  9-fasteoi acpi
12:         0            0           254            0            0       IO-APIC 12-edge    i8042
14:         0            0          55261            0            0       IO-APIC 14-edge    ata_piix
15:         0            0            0            0            0       IO-APIC 15-edge    ata_piix
16:         0            0            0            0            0       IO-APIC 16-fasteoi enp0s8
18:         0            0            0            0            0       IO-APIC 18-fasteoi vmwgfx
19:         0           1130            0            0            0       IO-APIC 19-fasteoi ehci_hcd:usb2, enp0s3
20:         0            0          1963            0            0       IO-APIC 20-fasteoi vboxguest
21:         0           2056            0            0            0       IO-APIC 21-fasteoi ahci[0000:00:0d.0], snd_intel8x0
22:         26            0            0            0            0       IO-APIC 22-fasteoi ohci_hcd:usb1
NMI:         0            0            0            0            0   Non-maskable interrupts
LOC:    231883          119552          120286          111275   Local timer interrupts
SPU:         0            0            0            0            0   Spurious interrupts
PMI:         0            0            0            0            0   Performance monitoring interrupts
IWI:        162           130           90           109   IRQ work interrupts
RTR:         0            0            0            0            0   APIC ICR read retries
RES:       1140          1412          1875          1869   Rescheduling interrupts
CAL:     53995          56472          38717          46359   Function call interrupts
TLB:       3196          3099          4291          3752   TLB shutdowns
TRM:         0            0            0            0            0   Thermal event interrupts
THR:         0            0            0            0            0   Threshold APIC interrupts
DFR:         0            0            0            0            0   Deferred Error APIC interrupts
MCE:         0            0            0            0            0   Machine check exceptions
MCP:         3            3            3            3            3   Machine check polls
ERR:         0
MIS:         0
PIN:         0            0            0            0            0   Posted-interrupt notification event
NPI:         0            0            0            0            0   Nested posted-interrupt event
PIW:         0            0            0            0            0   Posted-interrupt wakeup event

```

Muestra el mecanismo de E/S por interrupciones, detallando cuántas veces los controladores de hardware (como el teclado o el disco) han solicitado atención inmediata de la CPU. En términos de E/S, revela cómo los periféricos virtuales notifican al sistema que tienen datos listos para ser procesados de forma asíncrona y eficiente sin recurrir al sondeo constante.

2. cat /proc/iomem — rangos de memoria asignados a dispositivos (memory-mapped I/O)

```
ubxuser@Practica:~$ cat /proc/iomem
00000000-00000000 : Reserved
00000000-00000000 : System RAM
00000000-00000000 : Reserved
00000000-00000000 : PCI Bus 0000:00
00000000-00000000 : Video ROM
00000000-00000000 : Adapter ROM
00000000-00000000 : Reserved
00000000-00000000 : System ROM
00000000-00000000 : System RAM
00000000-00000000 : ACPI Tables
00000000-00000000 : PCI Bus 0000:00
00000000-00000000 : 0000:00:02.0
00000000-00000000 : vmwgfx probe
00000000-00000000 : 0000:00:02.0
00000000-00000000 : vmwgfx probe
00000000-00000000 : 0000:00:03.0
00000000-00000000 : e1000
00000000-00000000 : 0000:00:04.0
00000000-00000000 : vboxguest
00000000-00000000 : 0000:00:04.0
00000000-00000000 : 0000:00:06.0
00000000-00000000 : ohci_hcd
00000000-00000000 : 0000:00:08.0
00000000-00000000 : e1000
00000000-00000000 : 0000:00:09.0
00000000-00000000 : e1000
00000000-00000000 : 0000:00:0b.0
00000000-00000000 : ehci_hcd
00000000-00000000 : 0000:00:0d.0
00000000-00000000 : ahci
00000000-00000000 : Reserved
00000000-00000000 : IOAPIC 0
00000000-00000000 : Reserved
00000000-00000000 : Reserved
00000000-00000000 : System RAM
00000000-00000000 : Kernel code
00000000-00000000 : Kernel rodata
00000000-00000000 : Kernel data
00000000-00000000 : Kernel bss
00000000-00000000 : RAM buffer
ubxuser@Practica:~$
```

```
ubuntu@ubuntu:~$ cat /proc/iomem
00000000-00000000 : Reserved
00000000-00000000 : System RAM
00000000-00000000 : Reserved
00000000-00000000 : PCI Bus 0000:00
00000000-00000000 : Video ROM
00000000-00000000 : Adapter ROM
00000000-00000000 : Reserved
00000000-00000000 : System ROM
00000000-00000000 : System RAM
00000000-00000000 : ACPI Tables
00000000-00000000 : PCI Bus 0000:00
00000000-00000000 : 0000:00:02.0
00000000-00000000 : vmwgfx probe
00000000-00000000 : 0000:00:02.0
00000000-00000000 : vmwgfx probe
00000000-00000000 : 0000:00:03.0
00000000-00000000 : e1000
00000000-00000000 : 0000:00:04.0
00000000-00000000 : vboxguest
00000000-00000000 : 0000:00:04.0
00000000-00000000 : 0000:00:06.0
00000000-00000000 : ohci_hcd
00000000-00000000 : 0000:00:08.0
00000000-00000000 : e1000
00000000-00000000 : 0000:00:0b.0
00000000-00000000 : ehci_hcd
00000000-00000000 : 0000:00:0d.0
00000000-00000000 : ahci
00000000-00000000 : Reserved
00000000-00000000 : IOAPIC 0
00000000-00000000 : Reserved
00000000-00000000 : Reserved
00000000-00000000 : System RAM
00000000-00000000 : Kernel code
-----
```

Muestra el mapa de E/S mapeada en memoria (Memory-Mapped I/O), exponiendo las direcciones de la memoria RAM física que el sistema reserva para comunicarse con los controladores de hardware. A través de este espacio, la CPU puede enviar comandos o recibir datos de periféricos de alta velocidad (como la tarjeta de video o de red) utilizando instrucciones estándar de lectura y escritura en memoria.

3. cat /proc/ioports — puertos de E/S asignados a dispositivos (port-mapped I/O)

uboxuser@Practica:~\$

```
0000-0000 : PCT BUS 0000:00
```

```

0000-0000 : ata_piix
0000-0000 : PCI conf1
0000-0000 : PCI Bus 0000:00
0000-0000 : 0000:00:07.0
0000-0000 : ACPI PM1a_EVT_BLK
0000-0000 : ACPI PM1a_CNT_BLK
0000-0000 : ACPI PM_TMR
0000-0000 : ACPI GPE0_BLK
0000-0000 : 0000:00:07.0
0000-0000 : piix4_smbus
0000-0000 : 0000:00:01.1
0000-0000 : ata_piix
0000-0000 : 0000:00:02.0
0000-0000 : vmwgfx probe
0000-0000 : 0000:00:03.0
0000-0000 : e1000
0000-0000 : 0000:00:04.0
0000-0000 : vboxguest
0000-0000 : 0000:00:05.0
0000-0000 : Intel 82801AA-ICH
0000-0000 : 0000:00:05.0
0000-0000 : Intel 82801AA-ICH
0000-0000 : 0000:00:08.0
0000-0000 : e1000
0000-0000 : 0000:00:0d.0
0000-0000 : ahci
0000-0000 : 0000:00:0d.0
0000-0000 : ahci
0000-0000 : 0000:00:0d.0
0000-0000 : ahci
0000-0000 : 0000:00:0d.0
0000-0000 : ahci
0000-0000 : 0000:00:0d.0
0000-0000 : ahci
ubuntu@ubuntu:~$

```

Muestra las direcciones asignadas para la E/S mapeada por puertos (Port-Mapped I/O), las cuales conforman un espacio de direccionamiento completamente independiente y aislado de la memoria RAM. La CPU utiliza instrucciones de ensamblador dedicadas (IN y OUT) apuntando a estos puertos para controlar y consultar el estado de dispositivos de baja velocidad o heredados.

4. `cat /proc/devices` — major numbers de dispositivos de caracteres y de bloque registrados

```

108 ppp
116 alsa
128 ptm
136 pts
180 usb
189 usb_device
202 cpu/msr
204 ttyMAX
226 dma
241 hidraw
242 ttyDBC
243 bsg
244 watchdog
245 remoteproc
246 ptp
247 pps
248 rtc
249 dma_heap
250 dax
251 dimmctl
252 ndctl
253 tpm
254 gpiochip
261 accel

Block devices:
7 loop
8 sd
9 md
11 sr
65 sd
66 sd
67 sd
68 sd
69 sd
70 sd
71 sd
128 sd
129 sd
130 sd
131 sd
132 sd
133 sd
134 sd
135 sd
252 device-mapper
253 virtblk
254 ndp
259 blkext
ubuntu@Practica:~$

```

```

ubuntu@ubuntu:~$ cat /proc/devices
Character devices:
1 mem
4 /dev/vc/0
4 tty
4 ttyS
5 /dev/tty
5 /dev/console
5 /dev/ptmx
5 ttyprintk
6 lp
7 vcs
10 misc
13 input
21 sg
29 fb
89 i2c
99 ppdev
100 ppp
116 alsa
128 ptm
136 pts
180 usb
189 usb_device
202 cpu/msr
204 ttyMAX
226 dma
240 hidraw
241 ttyDBC
242 bsg
243 watchdog
244 remoteproc
245 ptp
246 pps
247 rtc

```

```

248 dma_heap
249 dax
250 dimmctl
251 ndctl
252 tpm
253 pwm
254 gpiochip
261 accel

Block devices:
 7 loop
 8 sd
 9 md
11 sr
65 sd
66 sd
67 sd
68 sd
69 sd
70 sd
71 sd
128 sd
129 sd
130 sd
131 sd
132 sd
133 sd
134 sd
135 sd
230 zvol
252 device-mapper
253 virtblk
254 mdp
259 blkext
ubuntu@ubuntu:~$

```

Enumera todos los controladores activos en el núcleo clasificados según su flujo de E/S en dispositivos de caracteres (flujos de bytes secuenciales) y de bloque (unidades de almacenamiento con acceso aleatorio). Muestra además el Major Number de cada uno, que es el identificador numérico que usa el kernel para asociar los archivos especiales en /dev con su respectivo driver de hardware.

5. lspci -v | head -80 — identificar controladores PCI de la VM y sus drivers en el kernel

```

Capabilities: <access denied>
Kernel driver in use: e1000
Kernel modules: e1000

00:04.0 System peripheral: ImoTek Systemberatung GmbH VirtualBox Guest Service
Flags: fast devsel, IRQ 20
I/O ports at d040 [size=32]
Memory at f0400000 (32-bit, non-prefetchable) [size=4M]
Memory at f0800000 (32-bit, prefetchable) [size=16K]
Kernel driver in use: vboxguest
Kernel modules: vboxguest

00:05.0 Multimedia audio controller: Intel Corporation 82801AA AC'97 Audio Controller (rev 01)
Subsystem: Dell 82801AA AC'97 Audio Controller
Flags: bus master, medium devsel, latency 64, IRQ 21
I/O ports at d180 [size=256]
I/O ports at d200 [size=64]
Kernel driver in use: snd_intel8x0
Kernel modules: snd_intel8x0

00:06.0 USB controller: Apple Inc. KeyLargo/Intrepid USB (prog-if 10 [OHCI])
Flags: bus master, fast devsel, latency 64, IRQ 22
Memory at f0804000 (32-bit, non-prefetchable) [size=4K]
Kernel driver in use: ohci-pci

00:07.0 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
Flags: medium devsel, IRQ 9
Kernel driver in use: piix4_smbus
Kernel modules: i2c_piix4

00:08.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
Flags: bus master, 66MHz, medium devsel, latency 64, IRQ 16
Memory at f0820000 (32-bit, non-prefetchable) [size=128K]
I/O ports at d240 [size=8]
Capabilities: <access denied>
Kernel driver in use: e1000
Kernel modules: e1000

00:09.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
Flags: bus master, 66MHz, medium devsel, latency 64, IRQ 17
Memory at f0840000 (32-bit, non-prefetchable) [size=128K]
I/O ports at d248 [size=8]
Capabilities: <access denied>
Kernel driver in use: e1000
Kernel modules: e1000

00:0b.0 USB controller: Intel Corporation 82801FB/FBM/FR/FW/FRW (ICH6 Family) USB2 EHCI Controller (prog-if 20 [EHCI])
vboxuser@Practica1:~$

ubuntu@ubuntu:~$ lspci -v | head -80
00:00.0 Host bridge: Intel Corporation 440FX - 82441FX PMC [Natoma] (rev 02)
Flags: fast devsel
Kernel modules: intel_agp

00:01.0 ISA bridge: Intel Corporation 82371SB PIIX3 ISA [Natoma/Triton II]
Flags: bus master, medium devsel, latency 0

00:01.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01) (prog-if 8a [ISA Compatibility mode controller, supports both channels switc
hed to PCI native mode, supports bus mastering])
Flags: bus master, fast devsel, latency 64
I/O ports at 01f0 [size=8]
I/O ports at 03fa [size=8]
I/O ports at 0170 [size=8]
I/O ports at 0374 [size=8]
I/O ports at d000 [size=16]
Kernel driver in use: ata_piix
Kernel modules: ata_acpi

00:02.0 VGA compatible controller: VMware SVGA II Adapter (prog-if 00 [VGA controller])
Subsystem: VMware SVGA II Adapter
Flags: bus master, fast devsel, latency 64, IRQ 18
I/O ports at d010 [size=16]
Memory at e0000000 (32-bit, prefetchable) [size=16M]
Memory at f0000000 (32-bit, non-prefetchable) [size=2M]
Expansion ROM at 000c0000 [virtual] [disabled] [size=128K]
Kernel driver in use: vmwgfx
Kernel modules: vmwgfx

00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
Flags: bus master, 66MHz, medium devsel, latency 64, IRQ 19
Memory at f0200000 (32-bit, non-prefetchable) [size=128K]
I/O ports at d020 [size=8]
Capabilities: <access denied>

```

```
Kernel driver in use: e1000
Kernel modules: e1000

00:04.0 System peripheral: Intel Corporation 82801AA VirtualBox Guest Service
Flags: fast devsel, IRQ 20
I/O ports at d040 [size=32]
Memory at f0000000 (32-bit, non-prefetchable) [size=4M]
Memory at f0000000 (32-bit, prefetchable) [size=16K]
Kernel driver in use: vboxguest
Kernel modules: vboxguest

00:05.0 Multimedia audio controller: Intel Corporation 82801AA AC'97 Audio Controller (rev 01)
Subsystem: Dell Device 0177
Flags: bus master, medium devsel, latency 64, IRQ 21
I/O ports at d100 [size=256]
I/O ports at d200 [size=64]
Kernel driver in use: snd_intel8x0
Kernel modules: snd_intel8x0

00:06.0 USB controller: Apple Inc. KeyLargo/Intrepid USB (prog-if 10 [OHCI])
Flags: bus master, fast devsel, latency 64, IRQ 22
Memory at f0004000 (32-bit, non-prefetchable) [size=4K]
Kernel driver in use: ohci-pci
Kernel modules: ohci-pci

00:07.0 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
Flags: medium devsel, IRQ 9
Kernel driver in use: piix4_smbus
Kernel modules: i2c_piix4

00:08.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
Flags: bus master, fast devsel, medium devsel, latency 64, IRQ 10
Memory at f0020000 (32-bit, non-prefetchable) [size=128K]
I/O ports at d240 [size=8]
```

Muestra el estado y los recursos de E/S asignados a los dispositivos conectados a la autopista principal de hardware, el bus PCI. Expone dinámicamente qué rangos de memoria, puertos e interrupciones usa cada componente emulado por el hipervisor, junto con el driver del kernel encargado de gestionar su transferencia de datos.

C. Kernel Module Individual (Tema 5.3)

Cada integrante escribe y compila su propio módulo. La especificación exacta está en la Sección 3 de este documento (es diferente para cada pareja).

Requisitos mínimos comunes para todos los módulos:

- Usar `MODULE_AUTHOR("[nombre completo]")`, `MODULE_DESCRIPTION` y `MODULE_LICENSE("GPL")`
- La función `init_module` debe imprimir con `printk(KERN_INFO, ...)` datos reales del subsistema asignado
- La función `cleanup_module` debe imprimir un mensaje de descarga con el nombre del autor
- Compilar con el Makefile estándar y demostrar carga con `insmod + dmesg` y descarga con `rmmod + dmesg`

1. Instalar cabeceras del kernel y herramientas de compilación

```
sudo apt install linux-headers-$(uname -r) build-essential -y
```

2. Crear Makefile con: `obj-m := nombre_modulo.o`

3. Compilar

```
make -C /lib/modules/$(uname -r)/build M=$(pwd) modules
```

4. Cargar y verificar mensaje `printk`

```
sudo insmod nombre_modulo.ko && dmesg | tail -8
```

5. Verificar que aparece en la lista de módulos

```
lsmod | grep nombre_modulo
```

6. Descargar y verificar mensaje de `cleanup`

```
sudo rmmod nombre_modulo && dmesg | tail -5
```

```

GNU nano 7.2                                     Makefile
obj-m:=modulo_gabriel.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) clean

```

```

GNU nano 7.2                                     modulo_gabriel.c
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h> //Crucial para copy_to_user

#define DEVICE_NAME "io_chardev_Alejos"
#define AUTHOR_NAME "Gabriel_Alejos"

MODULE_AUTHOR(AUTHOR_NAME);
MODULE_DESCRIPTION("MODULO DE DISPOSITIVO DE CARACTERES CON MAJOR DINAMICO");
MODULE_LICENSE("GPL");

static int major_number;
static char message[256];
static short message_size;

//Funcion de lectura del dispositivo

static ssize_t dev_read(struct file *filep, char __user *buffer, size_t len, loff_t *offset) {
    int uncopied_bytes = 0;

    //Si el offset ya alcanza o supera el tamaño del mensaje, enviamos EOF (0)
    if(*offset >= message_size) {
        return 0;
    }

    //Ajustamos la longitud para no leer fuera de los límites de nuestro string
    if(len > message_size - *offset) {
        len = message_size - *offset;
    }

    //copy_to_user(destino, origen, cantidad) transfiere los datos al espacio de usuario
    uncopied_bytes = copy_to_user(buffer, message + *offset, len);

    if(uncopied_bytes == 0){
        *offset += len; //actualizamos el offset
        return len; //retornamos los bytes leídos
    }else{
        return -EFAULT; //Error de acceso a la memoria
    }
}

//Estructura de operaciones del archivo

static struct file_operations fops = {
    .read = dev_read,
};

//Funcion de inicializacion

int init_module(void){
    //Registrar el dispositivo con Major 0 para que el kernel asigne uno dinamico
    major_number = register_chrdev(0, DEVICE_NAME, &fops);

    if(major_number < 0){
        printk(KERN_ALERT "FALLO AL REGISTRAR EL DISPOSITIVO %s CON ERROR %d\n", DEVICE_NAME, major_number);
        return major_number;
    }

    //Preparamos el mensaje que read() devolviera mas adelante
    message_size = sprintf(message, "Autor: %s | Major: %d\n", AUTHOR_NAME, major_number);

    //Imprimir datos reales del subsistema (major devuelto)
    printk(KERN_INFO "dispositivo '%s' registrado. \n", DEVICE_NAME);
    printk(KERN_INFO "major dinamico asignado por el subsistema: %d\n", major_number);

    return 0;
}

//Funcion de limpieza/descarga

void cleanup_module(void){
    unregister_chrdev(major_number, DEVICE_NAME);
    printk(KERN_INFO "Modulo descargado. Autor: %s\n", AUTHOR_NAME);
}

```

```

GNU nano 7.2                                     modulo_gabriel.c
}

//Estructura de operaciones del archivo

static struct file_operations fops = {
    .read = dev_read,
};

//Funcion de inicializacion

int init_module(void){
    //Registrar el dispositivo con Major 0 para que el kernel asigne uno dinamico
    major_number = register_chrdev(0, DEVICE_NAME, &fops);

    if(major_number < 0){
        printk(KERN_ALERT "FALLO AL REGISTRAR EL DISPOSITIVO %s CON ERROR %d\n", DEVICE_NAME, major_number);
        return major_number;
    }

    //Preparamos el mensaje que read() devolviera mas adelante
    message_size = sprintf(message, "Autor: %s | Major: %d\n", AUTHOR_NAME, major_number);

    //Imprimir datos reales del subsistema (major devuelto)
    printk(KERN_INFO "dispositivo '%s' registrado. \n", DEVICE_NAME);
    printk(KERN_INFO "major dinamico asignado por el subsistema: %d\n", major_number);

    return 0;
}

//Funcion de limpieza/descarga

void cleanup_module(void){
    unregister_chrdev(major_number, DEVICE_NAME);
    printk(KERN_INFO "Modulo descargado. Autor: %s\n", AUTHOR_NAME);
}

```

```

vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ make
make -C /lib/modules/6.8.0-124-generic/build M=/home/vboxuser/kernel_module/Gabriel_Alejos modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-124-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ make clean
make -C /lib/modules/6.8.0-124-generic/build M=/home/vboxuser/kernel_module/Gabriel_Alejos clean
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-124-generic'
CLEAN /home/vboxuser/kernel_module/Gabriel_Alejos/Module.symvers
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ ls
Makefile modulo_gabriel.c
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ ls -l
total 8
-rw-r--r-- 1 vboxuser vboxuser 177 Jun  6 18:52 Makefile
-rw-r--r-- 1 vboxuser vboxuser 2153 Jun  6 19:07 modulo_gabriel.c
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ make clean
make -C /lib/modules/6.8.0-124-generic/build M=/home/vboxuser/kernel_module/Gabriel_Alejos clean
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-124-generic'
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ make
make -C /lib/modules/6.8.0-124-generic/build M=/home/vboxuser/kernel_module/Gabriel_Alejos modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-124-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] /home/vboxuser/kernel_module/Gabriel_Alejos/modulo_gabriel.o
MODPOST /home/vboxuser/kernel_module/Gabriel_Alejos/Module.symvers
CC [M] /home/vboxuser/kernel_module/Gabriel_Alejos/modulo_gabriel.mod.o
LD [M] /home/vboxuser/kernel_module/Gabriel_Alejos/modulo_gabriel.ko
BTF [M] /home/vboxuser/kernel_module/Gabriel_Alejos/modulo_gabriel.ko
Skipping BTF generation for /home/vboxuser/kernel_module/Gabriel_Alejos/modulo_gabriel.ko due to unavailability of umlink
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$

```

Makefile, modulo.c y compilación del kernel module de Gabriel Alejos

```
vboxuser@Practica: ~/kernel_module/Gabriel_Alejos$ sudo insmod modulo_gabriel.ko && dmesg | tail -8
dmesg: read kernel buffer failed: Operation not permitted
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ sudo dmesg | tail -8
[ 6570.178602] audit: type=1400 audit(1780771253.748:233): apparmor="DENIED" operation="mknod" class="file" profile="ubuntu_pro_esn_cache" name="/usr/lib/python3/dist-packages/uclient/_pycache/_secret_manager.cpython-312.pyc.139853267540016" pid=5502 comm="python3" requested_mask="c" denied_mask="c" fsuid=0 ouid=0
[ 6570.187039] audit: type=1400 audit(1780771253.748:234): apparmor="DENIED" operation="mknod" class="file" profile="ubuntu_pro_esn_cache" name="/usr/lib/python3/dist-packages/uclient/_pycache/_system.cpython-312.pyc.139853267542064" pid=5502 comm="python3" requested_mask="c" denied_mask="c" fsuid=0 ouid=0
[ 6570.210608] audit: type=1400 audit(1780771253.773:235): apparmor="DENIED" operation="mknod" class="file" profile="ubuntu_pro_apt_news" name="/usr/lib/python3/dist-packages/uclient/_pycache/_event_logger.cpython-312.pyc.129388479354544" pid=5501 comm="python3" requested_mask="c" denied_mask="c" fsuid=0 ouid=0
[ 6570.214042] audit: type=1400 audit(1780771253.764:236): apparmor="DENIED" operation="mknod" class="file" profile="ubuntu_pro_esn_cache" name="/usr/lib/python3/dist-packages/uclient/_pycache/_exceptions.cpython-312.pyc.139853268004400" pid=5502 comm="python3" requested_mask="c" denied_mask="c" fsuid=0 ouid=0
[ 6570.214049] audit: type=1400 audit(1780771253.784:237): apparmor="DENIED" operation="mknod" class="file" profile="ubuntu_pro_apt_news" name="/usr/lib/python3/dist-packages/uclient/_pycache/_yaml.cpython-312.pyc.129388468062384" pid=5501 comm="python3" requested_mask="c" denied_mask="c" fsuid=0 ouid=0
[ 6572.466091] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 1058334635 wd_nsec: 1058333907
[ 9502.098446] dispositivo 'io_chardev_Alejos' registrado.
[ 9502.098450] major dinamico asignado por el subsistema: 240
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$
```

```
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ lsmod | grep modulo_gabriel
modulo_gabriel      12288  0
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$
```

```
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ sudo dmesg | tail -n 5
sudo: dmesg: command not found
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$ sudo dmesg | tail -n 5
[ 6570.214049] audit: type=1400 audit(1780771253.784:237): apparmor="DENIED" operation="mknod" class="file" profile="ubuntu_pro_apt_news" name="/usr/lib/python3/dist-packages/uclient/_pycache/_yaml.cpython-312.pyc.129388468062384" pid=5501 comm="python3" requested_mask="c" denied_mask="c" fsuid=0 ouid=0
[ 6572.466091] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 1058334635 wd_nsec: 1058333907
[ 9502.098446] dispositivo 'io_chardev_Alejos' registrado.
[ 9502.098450] major dinamico asignado por el subsistema: 240
[ 9774.425573] Modulo descargado. Autor: Gabriel_Alejos
vboxuser@Practica:~/kernel_module/Gabriel_Alejos$
```

Comprobación de printk, existencia del modulo y descarga del modulo Gabriel Alejos

C. Kernel Module Individual (Tema 5.3) Josue Vilchez

sudo apt install linux-headers-\$(uname -r) build-essential -y

```
ubuntu@ubuntu:~$ sudo apt install linux-headers-$(uname -r) build-essential -y
linux-headers-7.0.0-14-generic is already the newest version (7.0.0-14.14).
linux-headers-7.0.0-14-generic set to manually installed.
build-essential is already the newest version (12.12ubuntu2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
ubuntu@ubuntu:~$
```

2. Crear Makefile con: `obj-m := nombre_modulo.o`

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ sudo nano Makefile
```

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ sudo nano modulo_josue
```

`make -C /lib/modules/$(uname -r)/build M=$(pwd) modules`

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ sudo make -C /lib/modules/$(uname -r)/build M=$(pwd) modules
make: Entering directory '/usr/src/linux-headers-7.0.0-14-generic'
make[1]: Entering directory '/home/ubuntu/Downloads/kernel-modulejosue'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
  You are using:          gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
warning: pahole version differs from the one used to build the kernel
  The kernel was built with: 131
  You are using:          0
CC [M]  modulo_josue.o
MODPOST Module.symvers
CC [M]  modulo_josue.mod.o
CC [M]  .module-common.o
LD [M]  modulo_josue.ko
BTF [M] modulo_josue.ko
Skipping BTF generation for modulo_josue.ko due to unavailability of vmlinux
make[1]: Leaving directory '/home/ubuntu/Downloads/kernel-modulejosue'
make: Leaving directory '/usr/src/linux-headers-7.0.0-14-generic'
```

```
sudo insmod nombre_modulo.ko && dmesg | tail -8
```

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ sudo insmod modulo_josue.ko && dmesg | tail -8
dmesg: read kernel buffer failed: Operation not permitted
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$
```

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ sudo dmesg | tail -8
[ 2774.286787] audit: type=1107 audit(1780853921.996:159): pid=1965 uid=995 auid=4294967295 ses=4294967295 subj=unconfined msg='apparmor="DENIED" operation="dbus_method_call" bus="system" path="/org/freedesktop/timedate1" interface="org.freedesktop.DBus.Properties" member="GetAll" mask="send" name=":1.139" pid=8764 label="snap.thunderbird.thunderbird" peer_pid=9028 peer_label="unconfined" exe="/usr/bin/dbus-daemon" sauid=995 hostname=? addr=? terminal=?'
[ 2775.029268] audit: type=1400 audit(1780853922.734:160): apparmor="DENIED" operation="open" class="file" profile="snap.thunderbird.thunderbird" name="/sys/devices/virtual/dmi/id/product_sku" pid=8764 comm="thunderbird-bin" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 3560.030483] capability: warning: 'gvfsd-admin' uses 32-bit capabilities (legacy support in use)
[ 4060.322957] audit: type=1400 audit(1780855208.030:161): apparmor="DENIED" operation="open" class="file" profile="snap.firmware-updater.firmware-notifier" name="/proc/sys/vm/max_map_count" pid=11795 comm="firmware-notifi" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 5664.256271] modulo_josue: module verification failed: signature and/or required key missing - tainting kernel
[ 5664.265117] iochardevVilchez: Subistema de dispositivo de caracteres registrado correctamente.
[ 5664.265120] iochardevVilchez: Dispositivo asignado al Major dinámico: 239
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$
```

```
lsmod | grep nombre_modulo
```

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ lsmod | grep modulo_josue
modulo_josue                16384  0
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$
```

```
sudo rmmod nombre_modulo && dmesg | tail -5
```

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ sudo rmmod modulo_josue.c && dmesg | tail -5
dmesg: read kernel buffer failed: Operation not permitted
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$
```

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ sudo dmesg | tail -5
[ 5664.265120] iochardevVilchez: Dispositivo asignado al Major dinámico: 239
[ 6204.182965] Modulo iochardevVilchez descargado correctamente. Autor: Josue Vilchez.
[ 6357.121698] iochardevVilchez: Subistema de dispositivo de caracteres registrado correctamente.
[ 6357.121703] iochardevVilchez: Dispositivo asignado al Major dinámico: 239
[ 6387.876187] Modulo iochardevVilchez descargado correctamente. Autor: Josue Vilchez.
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$
```

D. Emulación vs Paravirtualización en VirtualBox (Temas 6.1 y 6.2)

1. Ejecutar `lspci -v` dentro de la VM e identificar qué dispositivos muestran vendor 'InnoTek'/'Oracle' (emulados) y cuáles son 'VirtIO' (paravirtualizados)

```
Jun 7 19:01
ubuntu@ubuntu:~/Downloads/kernel-modulejosue
~/Downloads/kernel-modulejosue

ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ lspci -v
00:00.0 Host bridge: Intel Corporation 440FX - 82441FX PMC [Natoma] (rev 02)
    Flags: fast devsel
    Kernel modules: intel_agp

00:01.0 ISA bridge: Intel Corporation 82371SB PIIX3 ISA [Natoma/Triton II]
    Flags: bus master, medium devsel, latency 0

00:01.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01) (prog-if 8a [ISA Compatibility mode controller, supports both channels switched to PCI native mode, supports bus mastering])
    Flags: bus master, fast devsel, latency 64
    I/O ports at 01f0 [size=8]
    I/O ports at 03f4 [size=8]
    I/O ports at 0170 [size=8]
    I/O ports at 0374 [size=8]
    I/O ports at d000 [size=16]
    Kernel driver in use: ata_piix
    Kernel modules: pata_acpi

00:02.0 VGA compatible controller: VMware SVGA II Adapter (prog-if 00 [VGA controller])
    Subsystem: VMware SVGA II Adapter
    Flags: bus master, fast devsel, latency 64, IRQ 18
    I/O ports at d010 [size=16]
    Memory at e0000000 (32-bit, prefetchable) [size=16M]
    Memory at f0000000 (32-bit, non-prefetchable) [size=2M]
    Expansion ROM at 000c0000 [virtual] [disabled] [size=128K]
    Kernel driver in use: vmwgfx
    Kernel modules: vmwgfx

00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
    Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
```

```

00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
  Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
  Flags: bus master, 66MHz, medium devsel, latency 64, IRQ 19
  Memory at f0200000 (32-bit, non-prefetchable) [size=128K]
  I/O ports at d020 [size=8]
  Capabilities: <access denied>
  Kernel driver in use: e1000
  Kernel modules: e1000

00:04.0 System peripheral: InnoTek Systemberatung GmbH VirtualBox Guest Service
  Flags: fast devsel, IRQ 20
  I/O ports at d040 [size=32]
  Memory at f0400000 (32-bit, non-prefetchable) [size=4M]
  Memory at f0800000 (32-bit, prefetchable) [size=16K]
  Kernel driver in use: vboxguest
  Kernel modules: vboxguest

00:05.0 Multimedia audio controller: Intel Corporation 82801AA AC'97 Audio Controller (rev 01)
  Subsystem: Dell Device 0177
  Flags: bus master, medium devsel, latency 64, IRQ 21
  I/O ports at d100 [size=256]
  I/O ports at d200 [size=64]
  Kernel driver in use: snd_intel8x0
  Kernel modules: snd_intel8x0

00:06.0 USB controller: Apple Inc. KeyLargo/Intrepid USB (prog-if 10 [OHCI])
  Flags: bus master, fast devsel, latency 64, IRQ 22
  Memory at f0804000 (32-bit, non-prefetchable) [size=4K]
  Kernel driver in use: ohci-pci
  Kernel modules: ohci_pci

00:06.0 USB controller: Apple Inc. KeyLargo/Intrepid USB (prog-if 10 [OHCI])
  Flags: bus master, fast devsel, latency 64, IRQ 22
  Memory at f0804000 (32-bit, non-prefetchable) [size=4K]
  Kernel driver in use: ohci-pci
  Kernel modules: ohci_pci

00:07.0 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
  Flags: medium devsel, IRQ 9
  Kernel driver in use: piix4_smbus
  Kernel modules: i2c_piix4

00:08.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
  Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
  Flags: bus master, 66MHz, medium devsel, latency 64, IRQ 16
  Memory at f0820000 (32-bit, non-prefetchable) [size=128K]
  I/O ports at d240 [size=8]
  Capabilities: <access denied>
  Kernel driver in use: e1000
  Kernel modules: e1000

00:0b.0 USB controller: Intel Corporation 82801FB/FBM/FR/FW/FRW (ICH6 Family) USB2 EHCI Controller (prog-if 20 [EHCI])
  Flags: bus master, fast devsel, latency 64, IRQ 19
  Memory at f0840000 (32-bit, non-prefetchable) [size=4K]
  Kernel driver in use: ehci-pci
  Kernel modules: ehci_pci

00:0d.0 SATA controller: Intel Corporation 82801HM/HEM (ICH8M/ICH8M-E) SATA Controller [AHCI mode] (rev 02) (prog-if 01 [AHCI 1.0])
  Flags: bus master, fast devsel, latency 64, IRQ 21
  I/O ports at d248 [size=8]
  I/O ports at d250 [size=4]
  I/O ports at d258 [size=8]
  I/O ports at d260 [size=4]
  I/O ports at d270 [size=16]
  Memory at f0842000 (32-bit, non-prefetchable) [size=8K]
  Capabilities: <access denied>
  Kernel driver in use: ahci
  Kernel modules: ahci

Kernel modules: ohci_pci

00:07.0 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
  Flags: medium devsel, IRQ 9
  Kernel driver in use: piix4_smbus
  Kernel modules: i2c_piix4

00:08.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
  Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
  Flags: bus master, 66MHz, medium devsel, latency 64, IRQ 16
  Memory at f0820000 (32-bit, non-prefetchable) [size=128K]
  I/O ports at d240 [size=8]
  Capabilities: <access denied>
  Kernel driver in use: e1000
  Kernel modules: e1000

00:0b.0 USB controller: Intel Corporation 82801FB/FBM/FR/FW/FRW (ICH6 Family) USB2 EHCI Controller (prog-if 20 [EHCI])
  Flags: bus master, fast devsel, latency 64, IRQ 19
  Memory at f0840000 (32-bit, non-prefetchable) [size=4K]
  Kernel driver in use: ehci-pci
  Kernel modules: ehci_pci

00:0d.0 SATA controller: Intel Corporation 82801HM/HEM (ICH8M/ICH8M-E) SATA Controller [AHCI mode] (rev 02) (prog-if 01 [AHCI 1.0])
  Flags: bus master, fast devsel, latency 64, IRQ 21
  I/O ports at d248 [size=8]
  I/O ports at d250 [size=4]
  I/O ports at d258 [size=8]
  I/O ports at d260 [size=4]
  I/O ports at d270 [size=16]
  Memory at f0842000 (32-bit, non-prefetchable) [size=8K]
  Capabilities: <access denied>
  Kernel driver in use: ahci
  Kernel modules: ahci

ubuntu@ubuntu:~/Downloads/kernel-module$

```

Dispositivos identificados

```
00:04.0 System peripheral: InnoTek Systemberatung GmbH VirtualBox Guest Service
Flags: fast devsel, IRQ 20
I/O ports at d040 [size=32]
Memory at f0400000 (32-bit, non-prefetchable) [size=4M]
Memory at f0800000 (32-bit, prefetchable) [size=16K]
Kernel driver in use: vboxguest
Kernel modules: vboxguest
```

10 y 11 Al no poseer dispositivos virtlo(Paravirtualizados) no se puede responder las 2preguntas .

- 12 Explicar con evidencia propia del laboratorio: ¿por qué la paravirtualización requiere un driver especial en el SO huésped mientras que la emulación completa no?

En la emulación completa (o virtualización), el sistema operativo huésped no sabe que está corriendo en una máquina virtual. Cree que interactúa con componentes de hardware físicos y reales, VGA compatible controller: VMware SVGA II Adapter es un chip de video que nuestra vm piensa que tiene, cuando en verdad lo que tiene es un código emulándolo.

E1. Experimentos del Subsistema de E/S: E/S de Dispositivos de Caracteres (/dev)

- 5.1 — Ejecutar `ls -la /dev/ | grep -E '^[cb]' | head -30`. Clasificar 10 dispositivos en caracteres (c) vs bloque (b). Para cada uno anotar el número mayor (major) y menor (minor) y buscar su significado en `cat /proc/devices`

```
vboxuser@Practica:~$ sudo ls -la /dev/ | grep -E '^[cb]' | head -30
crw-r--r-- 1 root root 10, 235 Jun 8 20:29 autofs
crw-rw---- 1 root disk 10, 234 Jun 8 20:29 btrfs-control
crw----- 1 root root 5, 1 Jun 8 20:29 console
crw----- 1 root root 10, 260 Jun 8 20:29 cpu_dma_latency
crw----- 1 root root 10, 203 Jun 8 20:29 cuse
crw----- 1 root root 10, 258 Jun 8 20:29 ecryptfs
crw-rw---- 1 root video 29, 0 Jun 8 20:29 fb0
crw-rw-rw- 1 root root 1, 7 Jun 8 20:29 full
crw-rw-rw- 1 root root 10, 229 Jun 8 20:29 fuse
crw----- 1 root root 241, 0 Jun 8 20:29 hidraw0
crw----- 1 root root 10, 228 Jun 8 20:29 hpet
crw----- 1 root root 10, 183 Jun 8 20:29 hwrng
crw----- 1 root root 89, 0 Jun 8 20:29 i2c-0
crw-r--r-- 1 root root 1, 11 Jun 8 20:29 kmsg
brw-rw---- 1 root disk 7, 0 Jun 8 20:29 loop0
brw-rw---- 1 root disk 7, 1 Jun 8 20:29 loop1
brw-rw---- 1 root disk 7, 2 Jun 8 20:29 loop2
brw-rw---- 1 root disk 7, 3 Jun 8 20:29 loop3
brw-rw---- 1 root disk 7, 4 Jun 8 20:29 loop4
brw-rw---- 1 root disk 7, 5 Jun 8 20:29 loop5
brw-rw---- 1 root disk 7, 6 Jun 8 20:29 loop6
brw-rw---- 1 root disk 7, 7 Jun 8 20:29 loop7
crw-rw---- 1 root disk 10, 237 Jun 8 20:29 loop-control
crw----- 1 root root 10, 227 Jun 8 20:29 mcelog
crw-r----- 1 root kmem 1, 1 Jun 8 20:29 mem
crw-rw-rw- 1 root root 1, 3 Jun 8 20:29 null
crw----- 1 root root 10, 144 Jun 8 20:29 nvram
crw-r----- 1 root kmem 1, 4 Jun 8 20:29 port
crw----- 1 root root 108, 0 Jun 8 20:29 ppp
crw----- 1 root root 10, 1 Jun 8 20:29 psaux
vboxuser@Practica:~$
```

```
ubuntu@ubuntu:~/Downloads/kernel-module$ ls -la /dev/ | grep -E '^[cb]' | head -30
crw-r--r-- 1 root root 10, 235 Jun 7 16:52 autofs
crw-rw---- 1 root disk 10, 234 Jun 7 16:54 btrfs-control
crw----- 1 root root 5, 1 Jun 7 16:54 console
crw----- 1 root root 10, 260 Jun 7 16:52 cpu_dma_latency
crw----- 1 root root 10, 261 Jun 7 16:52 cpu_wakeup_latency
crw----- 1 root root 10, 203 Jun 7 16:52 cuse
crw----- 1 root root 10, 258 Jun 7 16:52 ecryptfs
crw-rw---- 1 root video 29, 0 Jun 7 16:52 fb0
crw-rw-rw- 1 root root 1, 7 Jun 7 16:52 full
crw-rw-rw- 1 root root 10, 229 Jun 7 16:52 fuse
crw----- 1 root root 240, 0 Jun 7 20:07 hidraw0
crw----- 1 root root 10, 228 Jun 7 16:52 hpet
crw----- 1 root root 10, 183 Jun 7 16:52 hwrng
crw----- 1 root root 89, 0 Jun 7 16:52 i2c-0
crw-r--r-- 1 root root 1, 11 Jun 7 16:52 kmsg
crw-rw---- 1 root disk 10, 237 Jun 7 16:52 loop-control
brw-rw---- 1 root disk 7, 0 Jun 7 17:15 loop0
brw-rw---- 1 root disk 7, 1 Jun 7 17:15 loop1
brw-rw---- 1 root disk 7, 10 Jun 7 17:15 loop10
brw-rw---- 1 root disk 7, 11 Jun 7 17:15 loop11
brw-rw---- 1 root disk 7, 12 Jun 7 17:15 loop12
brw-rw---- 1 root disk 7, 13 Jun 7 17:15 loop13
brw-rw---- 1 root disk 7, 14 Jun 7 17:15 loop14
brw-rw---- 1 root disk 7, 15 Jun 7 17:15 loop15
brw-rw---- 1 root disk 7, 16 Jun 7 17:15 loop16
brw-rw---- 1 root disk 7, 17 Jun 7 17:15 loop17
brw-rw---- 1 root disk 7, 2 Jun 7 17:15 loop2
brw-rw---- 1 root disk 7, 3 Jun 7 17:15 loop3
brw-rw---- 1 root disk 7, 4 Jun 7 17:15 loop4
brw-rw---- 1 root disk 7, 5 Jun 7 17:15 loop5
```

Nombre	Tipo c/b	Major	Minor
autofs	c (carácter)	10	235
console	c (carácter)	5	1
fb0	c (carácter)	29	0
full	c (carácter)	1	7
hidraw0	c (carácter)	241	0
loop0	b (bloque)	7	0
loop1	b (bloque)	7	1
mem	c (Carácter)	1	1
null	c (Carácter)	1	3
ppp	c (Carácter)	108	0

```

crw----- 1 root root 10, 260 Jun 7 16:52 cpu_dma_latency
crw----- 1 root root 10, 261 Jun 7 16:52 cpu_wakeup_latency
crw----- 1 root root 10, 203 Jun 7 16:52 cuse
crw----- 1 root root 10, 258 Jun 7 16:52 ecryptfs
crw-rw---- 1 root video 29, 0 Jun 7 16:52 fb0
crw-rw-rw- 1 root root 1, 7 Jun 7 16:52 full
crw-rw-rw- 1 root root 10, 229 Jun 7 16:52 fuse
crw----- 1 root root 240, 0 Jun 7 20:07 hidraw0
crw----- 1 root root 10, 228 Jun 7 16:52 hpet
crw----- 1 root root 10, 183 Jun 7 16:52 hwrng
crw----- 1 root root 89, 0 Jun 7 16:52 i2c-0
crw-r--r-- 1 root root 1, 11 Jun 7 16:52 kmsg
crw-rw---- 1 root disk 10, 237 Jun 7 16:52 loop-control
brw-rw---- 1 root disk 7, 0 Jun 7 17:15 loop0
brw-rw---- 1 root disk 7, 1 Jun 7 17:15 loop1
brw-rw---- 1 root disk 7, 10 Jun 7 17:15 loop10
brw-rw---- 1 root disk 7, 11 Jun 7 17:15 loop11
brw-rw---- 1 root disk 7, 12 Jun 7 17:15 loop12
brw-rw---- 1 root disk 7, 13 Jun 7 17:15 loop13
brw-rw---- 1 root disk 7, 14 Jun 7 17:15 loop14
brw-rw---- 1 root disk 7, 15 Jun 7 17:15 loop15
brw-rw---- 1 root disk 7, 16 Jun 7 17:15 loop16
brw-rw---- 1 root disk 7, 17 Jun 7 17:15 loop17
brw-rw---- 1 root disk 7, 2 Jun 7 17:15 loop2
brw-rw---- 1 root disk 7, 3 Jun 7 17:15 loop3
brw-rw---- 1 root disk 7, 4 Jun 7 17:15 loop4
brw-rw---- 1 root disk 7, 5 Jun 7 17:15 loop5
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$

```

2. 5.2 — Comparar velocidades: dd if=/dev/zero of=/dev/null bs=1M count=2000 vs dd if=/dev/urandom of=/dev/null bs=1M count=200. ¿Por qué /dev/urandom es dramáticamente más lento si ambos son dispositivos de caracteres sin hardware real?

```

vboxuser@Practica:~$ dd if=/dev/zero of=/dev/null bs=1M count=2000
2000+0 records in
2000+0 records out
2097152000 bytes (2.1 GB, 2.0 GiB) copied, 0.0551296 s, 38.0 GB/s
vboxuser@Practica:~$ dd if=/dev/urandom of=/dev/null bs=1M count=200
200+0 records in
200+0 records out
209715200 bytes (210 MB, 200 MiB) copied, 0.413877 s, 507 MB/s
vboxuser@Practica:~$

ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ dd if=/dev/zero of=/dev/null bs=1M count=2000
2000+0 records in
2000+0 records out
2097152000 bytes (2.1 GB, 2.0 GiB) copied, 0.180087 s, 12 GB/s
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ dd if=/dev/urandom of=/dev/null bs=1M count=200
200+0 records in
200+0 records out
209715200 bytes (210 MB, 200 MiB) copied, 1.33816 s, 157 MB/s
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$

```

/dev/urandom es más lento para realizar la misma tarea debido a que es un proceso limitado por la CPU presente dentro de nuestra computadora, esto significa que la CPU es la responsable directa del tiempo requerido para realizar la tarea. El motivo por el que esto ocurre es que este comando obliga a la CPU a ejecutar ciertos algoritmos criptográficos en tiempo real para poder simular la aleatoriedad, en palabras simples, mide la velocidad innata de nuestra CPU y no la velocidad de la transferencia de datos. No obstante, /dev/zero no sufre de este problema ya que este solo duplica ceros rápidamente y directamente dentro de la memoria.

3. 5.3 — Usar strace dd if=/dev/zero of=/tmp/s bs=4k count=1 2>&1 | grep -E 'open|read|write|close'. Identificar las syscalls generadas. ¿Qué devuelve read() cuando lee de /dev/zero? ¿Y write() cuando escribe a /dev/null?


```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$
```

embargo, /dev/null lo descarta inmediatamente en la memoria RAM y no los guarda en el disco duro

- documentar los atributos udev del dispositivo

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Jun 7 16:52 /dev/null
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ ls -l /sys/dev/char/1:3
lrwxrwxrwx 1 root root 0 Jun 7 21:32 /sys/dev/char/1:3 -> ../../devices/virtual/mem/null
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$
```

vboxuser@Practica:~\$

```

ubuntu@ubuntu:~/Downloads/kernel-modulejesue$ udevadm info /dev/random
P: /devices/virtual/mem/random
M: random
J: c1:8
U: mem
D: c 1:8
N: random
L: 0
E: DEVPATH=/devices/virtual/mem/random
E: DEVNAME=/dev/random
E: DEVMODE=0666
E: MAJOR=1
E: MINOR=8
E: SUBSYSTEM=mem

ubuntu@ubuntu:~/Downloads/kernel-modulejesue$

```

El symlink apunta al driver mem dentro de la ruta de dispositivos virtuales de nuestra máquina, esto significa que /dev/null no esta en el hardware si no que es un software gestionado por el subsistema de Linux.

Por otra parte, udevadm nos da la siguiente informacion:

P: Ubicación real dentro del árbol de dispositivos del kernel

M: Random

D: c mayor 1 y 8 menor

U: mem Pertenece al subsistema de gestión de memoria del kernel

E: devpath = ruta interna del kernel

E: devname = ruta del archivo de dispositivo expuesta al usuario

E: 0666 = permisos de lectura y escritura para todos los usuarios del sistema: dueños grupos y otros

E: mayor = 1 Asociado al driver de controladores mem

E: minor = 8 sub indetificador único para el generador de números aleatorios

E: subsystem = mem

5. 5.5 — Demostrar bloqueo de /dev/random: ejecutar `timeout 3 cat /dev/random | hexdump -C | head`. ¿Se bloquea? Repetir con /dev/urandom. Ejecutar `cat /proc/sys/kernel/random/entropy_aval` y explicar qué es la entropía del kernel

```

vboxuser@Practica:~$ timeout 3 cat /dev/random | hexdump -C | head
00000000 d5 f2 d2 4d 2e cb 1d 58 a7 8b a8 95 18 19 c4 a6 1...M...X.....
00000010 ad c7 bb 14 34 67 13 49 c7 dc 33 61 85 bf af c1 1...4g...l...3a....
00000020 3f 8f b3 e9 54 b4 49 87 ea 2b 85 1f 03 62 21 26 1?...T...+...b!Rl
00000030 b3 1c bc 34 f0 e1 5e f6 76 4d cb 9e 9d 13 24 92 1...A...^uM...$..l
00000040 b3 b4 22 e2 de f0 cd 48 76 b0 a5 39 42 25 c7 a8 1...H...9Bz...l
00000050 e6 ee e2 ae 10 16 18 22 98 4c ad cd a5 49 9b 50 1...L...P...l
00000060 44 82 bd 86 d4 29 14 b4 90 9b 1c 5a ba 26 12 6e 1D...Z...n...l
00000070 f0 48 67 79 96 e2 b0 98 a1 1c 96 2b 58 20 1f a5 1Hgy...X...l
00000080 26 ed 17 f6 7f 15 f7 55 84 aa 9d 71 48 71 25 0f 1&...U...qhqz...l
00000090 44 1d 71 a5 fa 58 01 ee 95 7d 2a 0f 9f 8e 22 b7 1D.q.X...)*...l
vboxuser@Practica:~$ timeout 3 cat /dev/urandom | hexdump -C | head
00000000 97 83 50 46 2e 08 3b 7f bd aa 32 3d 48 41 7e d6 1..PF.....2=HAl..l
00000010 b7 04 26 78 1e 6f 6f e8 e3 df 1e 78 e2 4c 18 8b 1..&x...oo...x.L..l
00000020 36 56 d2 59 3f f7 9e ce aa f0 07 22 64 a4 aa e1 16U.Y?...d...l
00000030 4b 0e 34 9a 75 b0 df 5d 6f 00 5e 5c ad c4 b4 35 1K.4.u...Jo...5l
00000040 5c 20 f9 69 b8 85 30 5b b0 c2 be 08 92 4b 2e 1b 1N...l...l...R...l
00000050 05 de 97 34 30 40 64 dd 3a a0 63 3a f0 78 74 3d 1...400d...c:xt=l
00000060 b3 7c 21 18 10 3d 06 08 f7 c8 e3 74 75 4b d9 09 1..f...=...tuR...l
00000070 c3 02 5b 27 81 f4 aa a4 d8 f8 14 25 88 70 42 f5 1..f'.....%pB...l
00000080 fb af 52 cc c7 9e 0f 30 45 8e 7c d9 e9 7c 13 9a 1..R...0E.l...l..l
00000090 ca 1a cc 3a f6 4c fe 09 d1 c5 23 d9 5c 9f a6 e7 1...:L...#.N...l
vboxuser@Practica:~$ cat /proc/sys/kernel/random/entropy_aval
256
vboxuser@Practica:~$ _

```

```

ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ timeout 3 cat /dev/random | hexdump -C | head
00000000 1f c1 23 ed 83 e4 b9 3f 96 c5 29 c3 bb 34 0c 38 |..#....?)..4.8|
00000010 5f 06 1f 20 3a 19 9d 93 f7 c7 5b c0 db 25 9e 7f |_.. :.....[...%|
00000020 ae 43 de 2d 18 23 f4 c2 d1 5b d2 01 6e a3 97 e8 |.C.-.#...[.n...|
00000030 2f 54 45 1c 0d 51 d5 b2 1b 5f b9 60 f0 83 06 2c |/TE.Q..._`...|
00000040 30 e5 84 a6 7a bb ea 0b 46 9c 21 37 5c 87 2e 77 |0...z..kF.17\..w|
00000050 a8 b6 93 44 04 fc 7f ec 3b a0 a2 26 e7 fc 49 05 |...D...;...&..I.|
00000060 f3 2d ff 2b f1 5d 72 df 81 dc 4b 65 f1 76 4f 99 |...+.]r...Ke.v0.|
00000070 66 18 4a 2e 95 a9 87 4f 8c 2c 48 c2 d4 57 d0 cd |f.J...O.,H..W..|
00000080 53 02 62 2a 15 8a 19 a2 b2 b1 99 e0 8c 24 be a4 |S.b*.....$.|
00000090 f6 93 2a 5b 46 eb bf ef b2 80 01 92 7f 84 8a c3 |..*[F.....|
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ timeout 3 cat /dev/urandom | hexdump -C | head
00000000 32 02 b9 39 a7 dd 86 32 22 5d 52 a8 ce 03 d2 79 |2..9...2"]R...y|
00000010 5a 22 af 74 d8 8e 04 b4 a2 05 50 19 9b d8 ec 62 |Z".t.....P....b|
00000020 15 73 14 71 8a 53 01 32 6f f2 ac a6 03 0d 7a c6 |.s.q.S.2o....z.|
00000030 df 9d 84 74 d6 77 ca 1a c4 e1 6b 47 54 78 6b d7 |...t.w...kGTxk.|
00000040 f8 54 4e ad 16 06 39 3e 27 91 5a 36 79 85 f8 2d |.TN...9>' .Z6y..-|
00000050 c9 7a d2 bc 74 0f 8c 96 b1 fe 27 ca 68 91 f3 e1 |.z..t.....'.h...|
00000060 be d7 48 63 77 3d 1d 68 c3 73 62 03 e7 1e 2b d5 |..HCw=.h.sb...+.|
00000070 52 e9 31 ec de 52 a8 5b ce c3 5d 38 08 3b 63 68 |R.1..R.[..]8.;ch|
00000080 9a 59 09 82 cd f3 66 68 b2 ff 2f c2 c5 50 88 97 |.Y....fh../..P..|
00000090 8a 5f 46 00 6e b9 ec 4f 64 24 f4 0e 3b 0d c1 59 |.F.n..Od$.;..Y|
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ cat /proc/sys/kernel/random/entropy_avail
256
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$

```

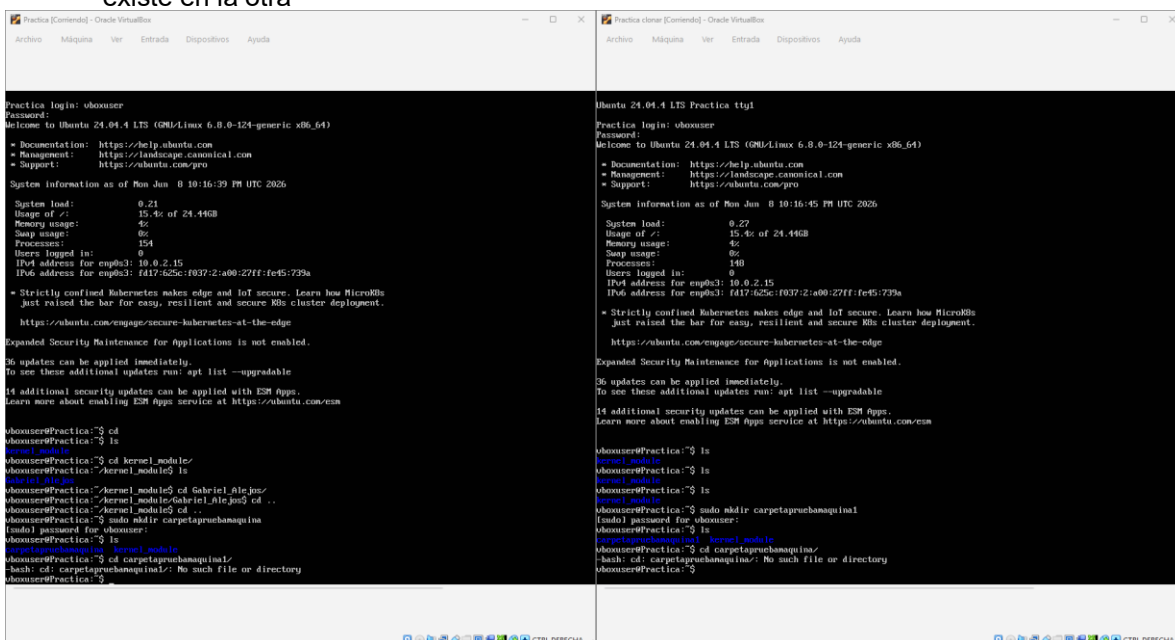
En este caso específico la maquina no se bloqueó, ambos comandos arrojaron sus datos respectivos de forma instantánea.

¿Qué es la entropía de kernel?

Es una medida de aleatoriedad que acumula el sistema operativo durante un periodo de tiempo, esta es imprescindible para la creación de claves criptográficas fuertes, contraseñas seguras, tokens SSH y certificados SSL que sean poco probables de adivinar o hackear. Esta aleatoriedad es acumulada mediante la medición de variaciones en microsegundos de eventos de hardware, como lo pueden ser el tiempo exacto entre pulsaciones de teclas o movimientos de mouse, el tiempo de respuesta de nuestro disco y las interrupciones de nuestra tarjeta de red al recibir paquetes de datos.

E2. Experimento de Virtualización: Full Clone vs Linked Clone en VirtualBox

- 6.1 — Crear un Full Clone de la VM (Machine > Clone > Full Clone). Medir el tiempo de clonación y el tamaño del archivo VDI. Arrancar ambas VMs simultáneamente y verificar que son completamente independientes creando un archivo en una y verificando que no existe en la otra



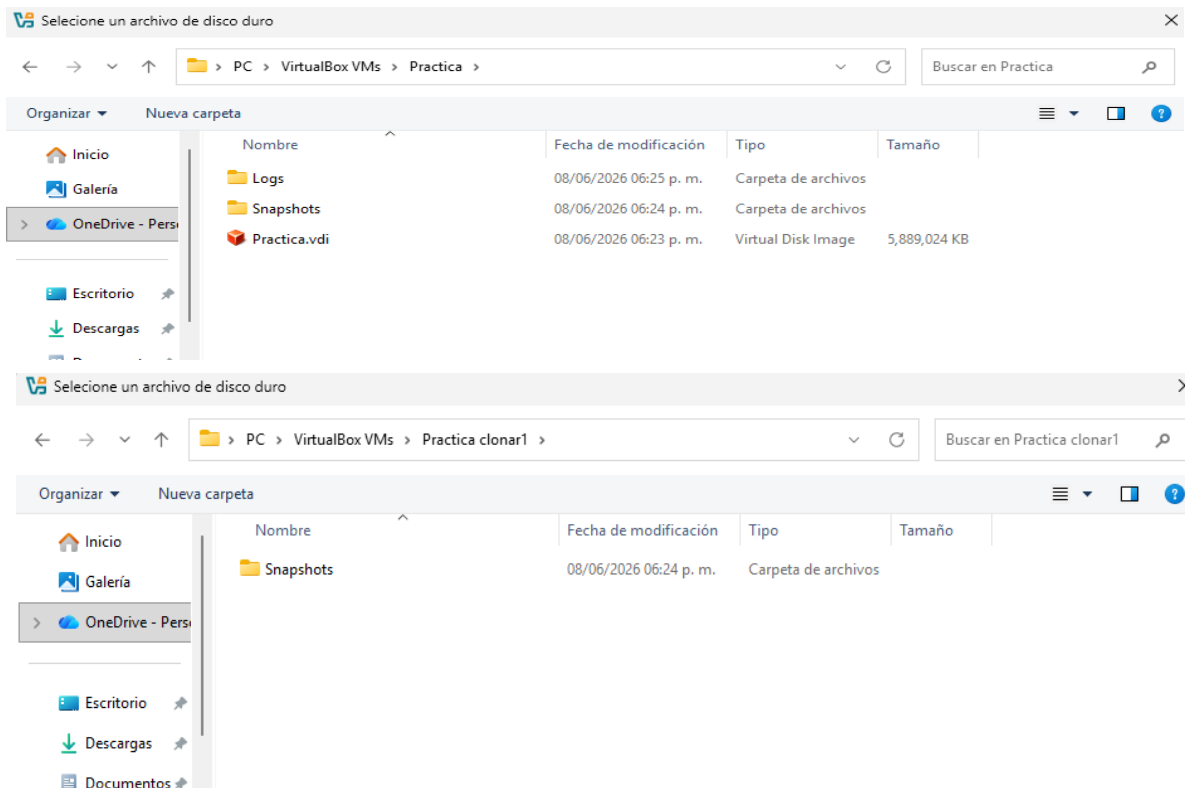
Como podemos apreciar, si intentamos entrar en las carpetas que creamos en la maquina virtual no es posible en la primera y viceversa, esto se debe a que solo mantiene la informacion que tenía la maquina anfitrión cuando la clonamos.

El tiempo de clonado tomo alrededor de unos 3 minutos, esto se debe a que la PC cuenta con sata SSD, además, el tamaño del archivo VDI de ambas son exactamente el mismo.

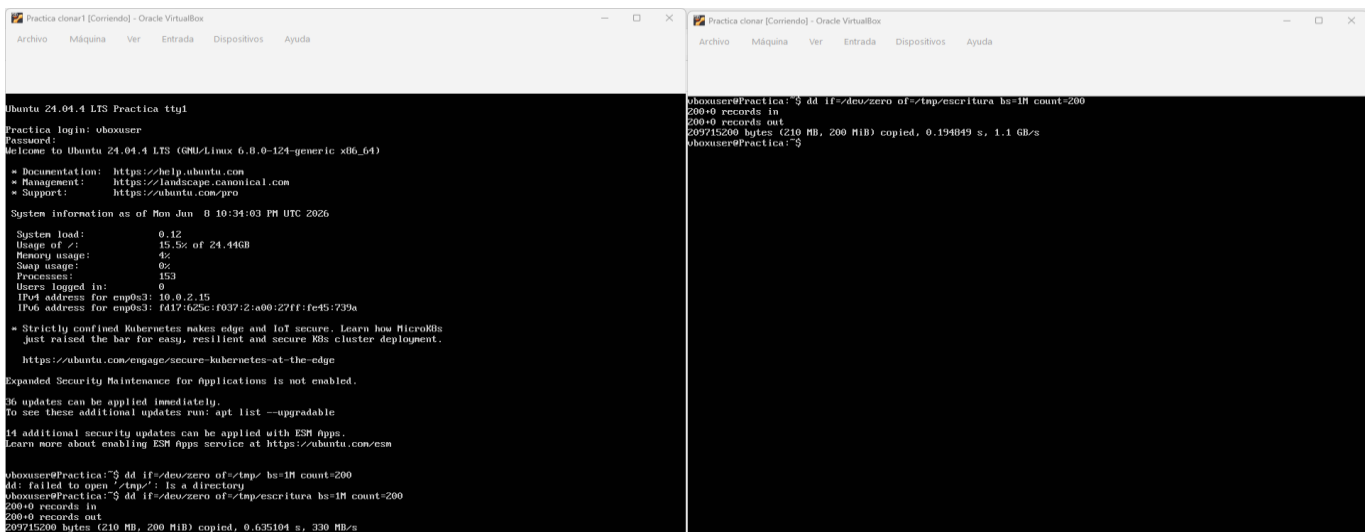
- 6.2 — Crear un Linked Clone de la misma VM base. Comparar el tamaño del VDI del Linked Clone vs el Full Clone. ¿Por qué el Linked Clone es mucho más pequeño inicialmente?

El linked clone tiene un peso diminuto comparado con el full clone, esto es debido a que se crea completamente vacío y solo guarda los cambios hechos dentro de esta misma maquina virtual linked.

Esto se puede apreciar mejor en esta imagen



- 6.2 — Con ambos clones corriendo, ejecutar en cada uno `dd if=/dev/zero of=/tmp/escritura bs=1M count=200`. Monitorear en el ANFITRIÓN con `iostat -ao` cuál genera más E/S de disco. Explicar por qué el Linked Clone puede generar más E/S debido al mecanismo Copy-on-Write



El linked clone genera más E/S que el anfitrión debido a que el mecanismo copy on write transforma una escritura en un proceso complejo de lectura, modificación y escritura que obliga al disco físico del anfitrión a leer constantemente el archivo base para verificar dependencias y luego escribir en el archivo, multiplicando el número de peticiones concurrentes de disco que pueden ser vistas desde iotop. Esto no ocurre en el full clone debido a que este escribe directamente dentro de su espacio independiente.

	Full Clone (Clon Completo)	Linked Clone (Clon Enlazado)
Tamaño Inicial en Disco	Ocupa exactamente el mismo espacio que la máquina virtual base (GB enteros de la máquina anfitrión).	Unos pocos MB o KB (solo el archivo de configuración y un disco diferencial vacío).
Tiempo de Creación	El hipervisor tiene que copiar el archivo.vdi original completo byte por byte en tu almacenamiento real. (alrededor de 3 minutos)	Solo toma una fracción de segundo, ya que el sistema solo crea un enlace lógico apuntando al disco base. (instantáneo)
E/S de Disco en el Anfitrión	El flujo de escritura es directo hacia su propio archivo.vdi. Es una operación lineal de una sola vía.	El mecanismo Copy on write obliga al disco del anfitrión a leer el bloque original del disco base antes de escribir el bloque modificado en el diferencial.
Rendimiento en Escritura Pesada	Al no tener que consultar dependencias externas, el flujo de datos es más constante y directo.	El procesamiento extra en el anfitrión genera una penalización en el rendimiento (como vimos en la prueba de 330 MB/s frente a 1.1 GB/s).

- Tabla clasificada de /dev con mayor/minor

Nombre del Nodo	Tipo	Número Mayor	Número Menor	Grupo	Descripción y Función en el Sistema
autofs	caracter	10	235	root	Controlador para el montaje

					automático de sistemas de archivos bajo demanda.
console	caracter	5	1	root	Consola del sistema del sistema. Retransmite los mensajes críticos del kernel durante el arranque.
fb0	caracter	29	0	video	<i>Framebuffer</i> de gráficos. Abstracción directa de la memoria de video para la interfaz de pantalla.
full	caracter	1	7	root	Archivo especial virtual. Retorna siempre un error de espacio insuficiente (ENOSPC) al intentar escribir.
fuse	caracter	10	229	root	Interfaz para <i>Filesystem in Userspace</i> . Permite ejecutar controladores de sistemas de archivos en espacio de usuario.
hwrng	caracter	10	183	root	<i>Hardware Random Number Generator</i> . Soporte físico para el reabastecimiento del pool de entropía del kernel.
i2c-0	caracter	89	0	root	Interfaz de comunicación para el primer bus de datos físico o emulado bajo protocolo.
loop0	bloque	7	0	disk	Primer dispositivo virtual de bucle. Mapea un archivo de datos regular, como si fuera un

					bloque de disco físico.
loop1	bloque	7	1	disk	Segundo dispositivo de bucle virtual del sistema.
loop10	bloque	7	10	disk	Dispositivo de bucle indexado con el identificador menor 10.
loop17	bloque	7	17	disk	Último nodo de la serie secuencial de bloques lógicos capturado en la salida de la terminal.

salida de strace mostrando syscalls de /dev/zero

Tipo de Syscall	Descriptor implicado	Bytes Procesados	Resultado / Estado
read	3	4096	Éxito (= 4096). El kernel satura el búfer del programa con puros caracteres nulos.
write	4	4096	Éxito (= 4096). El búfer se vuelca directamente al archivo en el disco/sistema de archivos temporal.
close	4	x	Éxito (= 0). Libera el descriptor del archivo destino.
close	3	x	Éxito (= 0). Libera el descriptor del dispositivo virtual.
openat	3	x	Éxito (= 3) Solicita al kernel abrir el archivo de origen. AT_FDCWD indica que la ruta se evalúa desde el directorio de trabajo actual.
openat	4	x	Éxito (= 4) Solicita al kernel la apertura/creación del archivo destino, devolviendo el siguiente descriptor libre en la tabla de procesos.

Pregunta Integradora — responder en el informe (mínimo 200 palabras con evidencia del laboratorio):

El driver de `/dev/zero` implementa `read()` devolviendo bytes nulos sin leer hardware real. El driver de `/dev/random` implementa `read()` bloqueando el proceso hasta tener entropía del hardware. Desde la perspectiva del modelo de E/S del kernel, ¿qué estructura de datos comparten ambos drivers para registrarse como dispositivos de caracteres? Explica el rol de `struct file_operations` y cómo el kernel despacha la llamada `read()` al driver correcto usando el número mayor del dispositivo.

Desde la perspectiva del modelo de E/S de Linux, los controladores de `/dev/zero` y `/dev/random` se registran compartiendo la estructura `struct cdev`. Esta estructura vincula el identificador único del dispositivo (`dev_t`) con una instancia de `struct file_operations`.

La estructura `struct file_operations` funciona como una interfaz orientada a objetos en C. Contiene un mapa de punteros a funciones que redirigen las llamadas del espacio de usuario (`openat`, `read`, etc.) hacia las rutinas reales del driver.

A pesar de compartir esta base, sus funciones asociadas operan de forma opuesta:

`/dev/zero`: Su puntero. `read` apunta a una rutina no bloqueante que inunda el búfer con bytes nulos (0x00) directamente en la RAM, alcanzando velocidades masivas en el comando `dd`.

`/dev/random`: Su puntero. `read` invoca una rutina criptográfica que verifica el pool de entropía (`entropy_avail`, con valor de 256 bits en nuestras capturas). Cuando este pool de entropía se vacía, el driver cambia el estado del proceso y entran en estado de espera, bloqueando el hilo de ejecución hasta que las interrupciones de hardware recolecten o aporten suficiente entropía. En nuestro caso la pausa no sucedió debido a que existía suficiente entropía.

En cuanto como el kernel despacha la llamada `read()` al driver correcto usando el número mayor del dispositivo, se basa en la arquitectura virtual file system (VFS) esta localiza su entrada en el sistema de archivo y accede a su nodo ya sea `c`(caracter) o `b`(bloque), después de esto consulta `cdev_map` en este punto es que el kernel usar el mayor extraído del nodo este identificara la clase general del driver podemos conocer lo que significa los mayor en nuestras `vm` gracias a `udevadm info`, pero solo con mayor tendremos una dirección general si queremos conocer el driver concreto tendremos que aplicar también menor.

```
ubuntu@ubuntu:~/Downloads/kernel-modulejosue$ udevadm info /dev/random
P: /devices/virtual/mem/random
M: random
J: c1:8
U: mem
D: c 1:8
N: random
L: 0
E: DEVPATH=/devices/virtual/mem/random
E: DEVNAME=/dev/random
E: DEVMODE=0666
E: MAJOR=1
E: MINOR=8
E: SUBSYSTEM=mem
```